Cache Bypassing Techniques

Sparsh Mittal

IIT Roorkee, India

Acronyms

- CBT = cache bypassing (CB) technique
- HCS = heterogeneous computing system
- NVM = non-volatile memory
- LLC = last level cache
- TLP = thread-level parallelism
- LRU = least recently used
- MSHR = miss status holding register

Motivation

- Memory demand is increasing fast.
- However, cache capacity cannot be increased proportionately
  - Caches already take ~30% chip area
  - Strict area/power-budgets
  - Many applications have poor-locality

- Idea: Selective cache bypassing (CB)
  - Increase “effective” cache capacity
  - Without incurring cost of larger sized cache

Exercise

- Is it possible that performance with cache can be worse than that without cache?

- YES. If cache hit rate is low, or there is cache contention.

Where cache bypassing may be useful?

Scope and Benefits of cache bypassing

- Blocks which show reuse in L1 cache may not show reuse in L2 cache
- With poor replacement policies, bypassing is very effective
- Bypassing reduces traffic to cache
  - Reduces writes in NVM cache
  - Allows aggressive SRAM cache reconfiguration
  - Reduces cache pollution in DRAM-cache which use large block size (e.g., 2KB)

Cache bypassing in GPUs

Li et al., SC 2015

	Processor	L1 cache	Thread/core	Cache/thread
CPUs	AMD Warsaw	16 KB	1	16KB
	Intel Haswell	32KB	2	16KB
	Intel Xeon-Phi	32KB	4	8KB
	Oracle M5	16KB	8	2KB
GPUs	Nvidia Fermi	48 KB	1536	32B
	Nvidia Kepler	48 KB	2048	24B
	Nvidia Maxwell	24 KB	2048	16B
	AMD Radeon-7	16 KB	2560	6.4B

So Small!

- GPUs share small cache between large number of threads => cache pollution
- Interference in L1D cache and in L1D-L2 link => major bottleneck in GPU performance
- Cache bypassing is even more important for GPUs

Challenges of Cache bypassing

Challenges of Cache Bypassing

- Naïve bypassing can increased miss-rate
- Bypassed requests go directly to the next level cache or memory=> Bandwidth congestion
- CPU and GPU characteristics different => need different policies for them
- Using CB in GPUs
  - may require thread-throttling or novel thread-scheduling policies
  - Reducing multithreading for improving cache utilization => compute resources under-utilized, harmful impact on algorithm

Challenges of Cache Bypassing

- In CPU-GPU systems:
  - Bypassing all GPU requests may increase the cache hit rate of CPU
  - However, it can degrade the performance of both CPU and GPU since huge number of bypassed GPU requests cause main memory contention

Background on CB: useful terms/concepts

Reuse distance

- **Reuse distance**: number of accesses seen by a cache set between two accesses to the same cache line

Access stream: P V Q T S W Z Y S T W V Q Z

← 6 distinct accesses →

Reuse distance of Q is 6.

- Keep a line in cache only until its expected reuse happens and cache pollution is avoided
- If reuse distance > associativity => bypass from cache

Live time and dead time

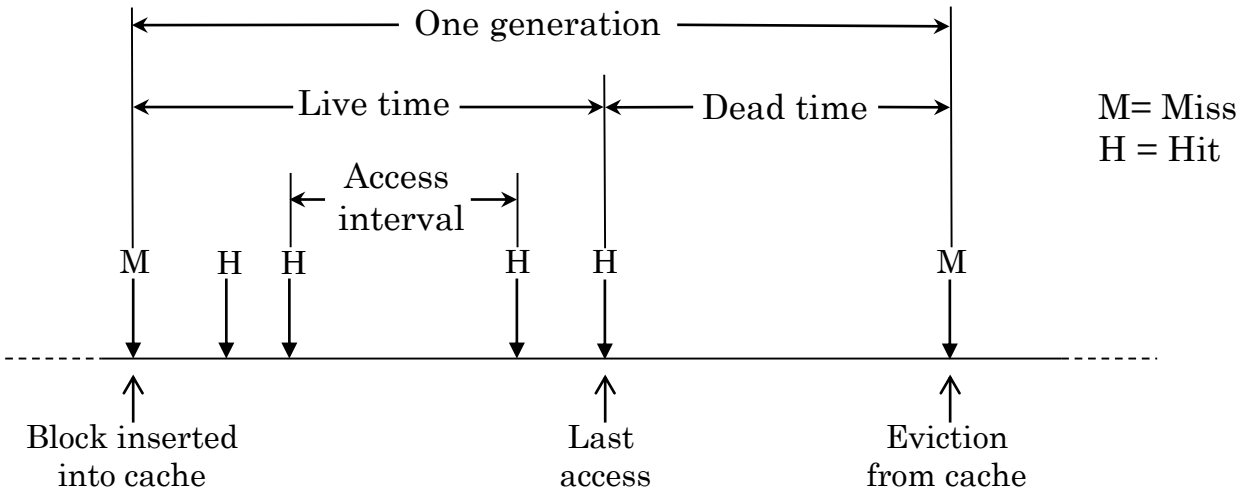


Illustration of one generation of a cache block

Different cache hierarchies

- A multi-level cache hierarchy is
  - **Inclusive**: if contents of all higher level caches are subset of LLC
  - **Non-inclusive:** when higher level caches may not be subsets of LLC.
  - **Exclusive:** no two cache levels have common cache contents.
- Using CB in inclusive cache hierarchy requires special provisions since CB violates inclusion

Which blocks to bypass?

Candidates for bypassing

- ‘dead on arrival’ block: a block with no reuse
- Blocks with reuse count < threshold1
- Blocks whose reuse distance > threshold2
- Thread-private data from shared cache
- Physical pages shared by multiple page tables

Dead block size

- Dead data can be predicted at granularity of:
  - Subblock (e.g., 8B)
  - Cache block (e.g., 64B)
  - Macroblock (e.g., 1KB)
  - Page (e.g., 4KB)
  - Superpage (2MB to 1GB)

How to find bypassing candidates (1/2)

- **Static:** Identify memory access pattern using compiler
  - Adjacent blocks show similar access properties, e.g., if P and Q are adjacent, then if P is dead block, Q is also a dead block
- **Runtime:** Use counters to record time and number of accesses to each block
  - E.g., count number of accesses to a block. If it reaches a threshold value => block has become dead
  - Behavior of a block in its previous generation
  - Reuse pattern of a block in one cache (L2) can guide CB decisions for this block in another cache (L3)

How to find bypassing candidates (2/2)

- Observation: Only few load instructions are responsible for majority of data cache misses.

Based on this:

- Measure miss rates of individual load/store instructions
- Bypass data references generated by instructions leading to highest miss rate

Another variant:

- Record misses that fetch useful (i.e., later reused) data into cache and misses that fetch dead-on-arrival data
- Bypass only latter category of data references

Reducing overhead of cache bypassing

Regulating aggressive of CB based on:

- Cache miss-rate
- Stacked DRAM bandwidth utilization
- NoC (network-on-chip) congestion
- Cache port obstruction
- Read/write energy of cache

Use of bypass buffer

- Use buffer to store both tags and/or data of bypassed blocks
- If a block found in this buffer => access to cache is avoided
- Move a block in buffer to cache only if it shows temporal reuse
- Accesses leading to capacity misses show short and temporary bursts of activity. Hence, bypass capacity misses and place them in a bypass buffer.

- With effective CB, size of buffer will be small

Reducing overhead of CB (1/2)

- Use set-sampling
- Use partial-tags (instead of full tags) since exact matching not required
- Predictors (e.g., dead-block predictors) indexed by PC of memory instructions incur less overhead than those indexed by addresses
- Sampling unit can have lower associativity than the main cache

Reducing overhead of CB (2/2)

- Bypass only when
  - no invalid block is available
- Prefetch data in dead blocks
- While replacing, give priority to dead blocks
- Use probabilistic bypassing : avoid predictor overhead

Probabilistic cache bypassing

Probabilistic bypassing with feedback

- Perform random bypassing based on a probability
- Increase/decrease probability based on whether bypassed line is accessed before the replacement victim
- To find this, use an additional tag and a competitor pointer with each set
- On a line bypass, store tag of bypassed line in this tag. Also, store replacement victim which would have been evicted without bypassing in competitor pointer
- If competitor accessed before bypassed tag => bypassing effective. And vice versa

Probabilistically finding hot blocks (1/2)

- Of the blocks comprising program working set
  - a few blocks are accessed very frequently and for the longest duration of time
  - remaining blocks are accessed infrequently and in bursty manner.
- Hence, set-associative caches serve majority of references from MRU position
- They effectively work as direct-mapped caches, while expending energy and latency of set-associative caches

Etsion et al. TC'12

Probabilistically finding hot blocks (2/2)

- Identify hot blocks by randomly selecting
  - Long-residency blocks have high chances of getting selected
- Use
  - a direct-mapped cache for serving hot blocks
  - a small fully-associative filter to serve transient blocks
- Based on this, hot blocks can be efficiently served and cold blocks can be bypassed

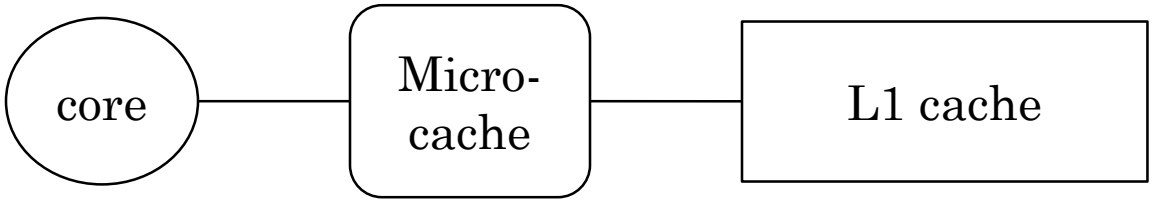
Etsion et al. TC'12

Some cache bypassing techniques

We will now discuss some CBTs to get insights into their working

Bypassing using compiler-assistance

- In statically scheduled processors, compiler is aware of the distance between a load and it reuse.
- Let L1 cache latency = T1 cycles



- Micro-cache should only store data that will be required before next T1 cycles
- Otherwise, bypass micro-cache and directly access L1.

Wu et al. MICRO'02

Bypassing based on cost/benefit analysis

- Cost of caching:
  - Time to access memory for fetching data
  - Doubled if caching a block replaces a dirty block.
- Benefit from caching:
  - Product of number of accesses to data during its cache residency and the difference between access time of memory and cache
- Bypass if **cost of caching > benefit of caching**

Chi et al. HICSS'89

Bypassing at sub-block granularity

- A large fraction of cache subblocks (e.g., 8 B subblock in a 64 B block) fetched are never or rarely reused
- Predict which sub-blocks will be referenced and how many times
- On a cache miss, only subblocks predicted-to-be-useful are fetched
- Turn-off a subblock after it has been referenced for expected number of times
- Preferentially evict a block with all dead sub-blocks

Cache bypassing in inclusive cache hierarchies

Bypassing violates inclusion property. To use CB in inclusive hierarchies, we need to satisfy inclusion properties

See survey paper for CB in exclusive hierarchies

Strategies to satisfy inclusion properties

- Insert bypassed blocks in LRU position
  - Block will be evicted on next miss to this set
  - Leads to competition at LRU position
  - Evicts one potentially useful block
- Use additional tags to track bypassed blocks
- Store bypassed blocks in a separate buffer
- Insert bypassed blocks at LRU position without writing their data & flag it as VOID.
  - Access to VOID block => miss, but coherence state bits of VOID blocks are updated as if they were valid
  - Maintains inclusion while reducing write-energy

Cache bypassing in context of NVM main memory

See survey paper for CB in DRAM caches

CB in context of NVM main memory

- Writing-back dirty data to NVM main memory incurs high overheads
- Frequently used data blocks are also frequently written-back data, so try to keep them in cache
- Partition each LLC set into a ‘frequent’ and an ‘infrequent’ writeback (WB) list.
- Find optimal size of each list based on miss penalty for clean and dirty blocks.
- If optimal size of frequent WB list = cache associativity, use set-sampling to check whether bypassing read requests from cache provides smaller number of misses than not bypassing them.

Wang et al. TACO'13

Cache bypassing in GPUs

CB in GPUs

- In GPUs, CB needs to be performed synergistically with a warp-scheduling scheme
- Evaluate impact of bypassing on GPU cores using core-sampling idea

- In GPUs, bypassing can be done based on reuse or memory divergence behavior
- Reuse-behavior: discussed earlier in this presentation

CB in GPUs

- **Memory divergence behavior:**
  - Making different cache/bypass decision for different threads of a warp creates differences in their latencies => all threads would be stalled till completion of last memory request
  - Make identical caching/bypassing decision for all threads of a warp. Cache few warps at a time
  - E.g., turn mostly-hit warps into all-hit, mostly-miss warps into all-miss warps
- Perform bypassing when no resources (e.g., MSHR) available for servicing a miss

Cache bypassing in CPU-GPU HCS with shared LLC

Bypassing based on GPU's cache sensitivity (1/2)

- GPU accesses can tolerate latency => bypass GPU requests in CPU-GPU HCS

- TLP of GPU = number of 'ready-to-schedule' warps
- High TLP => higher memory access latency can be tolerated by GPU

- => TLP shows cache sensitivity of an GPU app

Mekkat et al. PACT'13

Bypassing based on GPU's cache sensitivity (2/2)

- Bypass GPU requests in LLC if GPU has large TLP or is insensitive to LLC performance

- Assess impact of GPU bypassing on CPU performance using cache set-sampling. Use this to regulate aggressiveness of GPU bypassing

Mekkat et al. PACT'13

CB and memory channel-partitioning

- Challenge: GPU request bypassing improves CPU hit rate, but harms performance of both CPU and GPU due to main memory contention
- Solution: Partition memory channels b/w CPU and GPU
  - This improves CPU performance, esp. for memory-intensive workloads and offsets impact of reduced bandwidth
  - But reduces GPU performance since bandwidth requirement after LLC bypassing cannot be met with a single memory channel

Wang et al. VLSI-DAT'14

References

- S. Mittal, “A Survey Of Cache Bypassing Techniques”, JLPEA, 2016